

```
┌───────────────────────────────────────────────────────────────────────────────────┐ || AI Creative Studio POC
Backend || ┌───────────────────────────────────────────────────────────────────────────┐ Mode: MOCK Port:
3001 [DB] Schema initialized [DB] Seed user already exists, skipping ← OR "Seed user created"
[FlowProxy] Mode: MOCK [FlowProxy:MOCK] Ready — no credentials needed [Queue] Worker
```

started (concurrency=1) [SessionKeeper] Started — pinging Flow every 10 minutes  Server running at <http://localhost:3001> Test key: poc-test-key-12345678

Leave this terminal open. Do not close it.

PART 2 — BACKEND API TESTS (use browser or curl)

Open a NEW terminal. Keep the server running in the first one.

Use curl commands below OR open Postman/Insomnia and import manually.

TEST B-01: Health check

Request: GET <http://localhost:3001/health>

Expected response (200 OK): { "status": "ok", "version": "1.0.0-poc", "mode": "mock", "timestamp": "2026-...", "uptime": <number> }

curl command: curl <http://localhost:3001/health>

TEST B-02: Validate the built-in test key

Request: POST <http://localhost:3001/api/validate-key> Content-Type: application/json Body: { "license_key": "poc-test-key-12345678" }

Expected response (200 OK): { "valid": true, "user": { "id": 1, "email": "test@poc.local", "name": "Test User" }, "subscription": { "status": "active", "expires_at": "...", "days_remaining": <number>, "badge": "MONTHLY ACCESS" } }

curl command: curl -X POST <http://localhost:3001/api/validate-key>
-H "Content-Type: application/json"
-d '{"license_key":"poc-test-key-12345678"}'

TEST B-03: Validate with wrong key (must fail)

Request: POST <http://localhost:3001/api/validate-key> Body: { "license_key": "wrong-key-999" }

Expected response (401):

```
{  
  "valid": false,  
  "error": "INVALID_KEY",  
  "message": "Invalid or expired license key"  
}
```

```
curl command: curl -X POST http://localhost:3001/api/validate-key  
-H "Content-Type: application/json"  
-d '{"license_key":"wrong-key-999"}'
```

TEST B-04: Validate with empty key (must fail)

Request: POST http://localhost:3001/api/validate-key Body: { "license_key": "" }

Expected response (400):

```
{ "valid": false, "error": "MISSING_KEY" }
```

TEST B-05: Register a new user

Request: POST http://localhost:3001/api/register Content-Type: application/json Body: { "name": "Alice Test", "email": "alice@test.com" }

Expected response (201): { "license_key": "<32-character hex string>", "email": "alice@test.com", "user_id": 2, "message": "Registration successful" }

SAVE the license_key — you will use it in later tests.

```
curl command: curl -X POST http://localhost:3001/api/register  
-H "Content-Type: application/json"  
-d '{"name":"Alice Test","email":"alice@test.com"}'
```

TEST B-06: Register same email again (idempotent — must return same key)

Request: POST http://localhost:3001/api/register Body: { "name": "Alice Again", "email": "alice@test.com" }

Expected response (200): { "license_key": "<SAME key as TEST B-05>", "already_existed": true, "message": "Account already exists — here is your existing key" }

TEST B-07: Register with bad email (must fail)

Request: POST http://localhost:3001/api/register Body: { "name": "Bob", "email": "notanemail" }

Expected response (400):

```
{ "error": "INVALID_EMAIL" }
```

TEST B-08: Generate image — missing key header (must fail)

Request: POST <http://localhost:3001/api/generate/image> Content-Type: application/json Body: {
"prompt": "a sunset" } (NO X-License-Key header)

Expected response (401):

```
{ "error": "UNAUTHORIZED", "message": "Missing X-License-Key header" }
```

curl command: curl -X POST <http://localhost:3001/api/generate/image>
-H "Content-Type: application/json"
-d '{"prompt": "a sunset"}'

TEST B-09: Generate image — wrong key (must fail)

Request: POST <http://localhost:3001/api/generate/image> X-License-Key: fake-key-000 Body: {
"prompt": "a sunset" }

Expected response (401):

```
{ "error": "UNAUTHORIZED" }
```

curl command: curl -X POST <http://localhost:3001/api/generate/image>
-H "Content-Type: application/json"
-H "X-License-Key: fake-key-000"
-d '{"prompt": "a sunset"}'

TEST B-10: Generate image — no prompt (must fail)

Request: POST <http://localhost:3001/api/generate/image> X-License-Key: poc-test-key-12345678
Body: {}

Expected response (400):

```
{ "error": "MISSING_PROMPT" }
```

curl command: curl -X POST <http://localhost:3001/api/generate/image>
-H "Content-Type: application/json"
-H "X-License-Key: poc-test-key-12345678"
-d "{}"

TEST B-11: Generate image — MAIN HAPPY PATH

This is the most important test. Request: POST <http://localhost:3001/api/generate/image> X-License-Key: poc-test-key-12345678 Content-Type: application/json Body: { "prompt": "a glowing crystal forest at sunset", "aspect_ratio": "16:9" }

Expected response (202 — means queued): { "job_id": "<uuid like xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx>", "status": "queued", "type": "image", "message": "Image generation queued..." }

SAVE the job_id — use it in TEST B-13.

curl command: curl -X POST <http://localhost:3001/api/generate/image>
-H "Content-Type: application/json"
-H "X-License-Key: poc-test-key-12345678"
-d '{"prompt": "a glowing crystal forest at sunset", "aspect_ratio": "16:9"}'

Check server terminal — you should see: [Generate] Image job queued: <uuid> by user test@poc.local [Queue] Enqueued job <uuid> (BullMQ: bullmq-<uuid>) [Worker] Processing job <uuid> | type=image | prompt="a glowing crystal forest at sunset" [FlowProxy:MOCK] Image — "a glowing crystal forest at sunset" (4000ms) (wait 4 seconds) [FlowProxy:MOCK] Done → <https://picsum.photos/seed/ai.../1280/720> [Worker] Job <uuid> completed → <https://picsum.photos/seed/.../1280/720>

TEST B-12: Generate video — HAPPY PATH

Request: POST <http://localhost:3001/api/generate/video> X-License-Key: poc-test-key-12345678
Content-Type: application/json Body: { "prompt": "waves crashing on a rocky beach", "duration": 8, "quality": "fast" }

Expected response (202): { "job_id": "<uuid>", "status": "queued", "type": "video", "estimated_seconds": 90 }

SAVE this job_id too.

curl command: curl -X POST <http://localhost:3001/api/generate/video>
-H "Content-Type: application/json"
-H "X-License-Key: poc-test-key-12345678"
-d '{"prompt": "waves crashing on a rocky beach", "duration": 8, "quality": "fast"}'

TEST B-13: Poll job status (use job_id from TEST B-11)

Request: GET http://localhost:3001/api/jobs/<job_id from B-11> X-License-Key: poc-test-key-12345678

If called within 4 seconds of B-11:

{ "status": "processing", "is_done": false, "poll_again": true }

If called after 4 seconds: { "job_id": "...", "type": "image", "status": "completed", "output_url": "[https://picsum.photos/seed/aiX/1280/720](\"https://picsum.photos/seed/aiX/1280/720\")", "is_done": true, "poll_again": false }

curl command (replace JOB_ID): curl "http://localhost:3001/api/jobs/JOB_ID"
-H "X-License-Key: poc-test-key-12345678"

TEST B-14: Poll job from another user's key (must fail)

Use the key from TEST B-05 (alice's key) to poll a job created with the test key.

Request: GET http://localhost:3001/api/jobs/<job_id from B-11> X-License-Key: <alice's key from B-05>

Expected response (403):

{ "error": "FORBIDDEN", "message": "You do not have access to this job" }

TEST B-15: Poll non-existent job (must fail)

Request: GET <http://localhost:3001/api/jobs/00000000-0000-0000-0000-000000000000> X-License-Key: poc-test-key-12345678

Expected response (404):

{ "error": "JOB_NOT_FOUND" }

TEST B-16: List job history

Request: GET <http://localhost:3001/api/jobs> X-License-Key: poc-test-key-12345678

Expected response (200): { "jobs": [... list of your recent jobs ...], "count": <number> }

curl command: curl <http://localhost:3001/api/jobs>
-H "X-License-Key: poc-test-key-12345678"

TEST B-17: Generate video with invalid duration (must fail)

Request: POST <http://localhost:3001/api/generate/video> X-License-Key: poc-test-key-12345678
Body: { "prompt": "a cat", "duration": 999 }

Expected response (400):

{ "error": "INVALID_DURATION", "message": "duration must be one of: 4, 8, 16" }

TEST B-18: Generate video with invalid quality (must fail)

Request: POST <http://localhost:3001/api/generate/video> X-License-Key: poc-test-key-12345678

Body: { "prompt": "a cat", "quality": "ultra4k" }

Expected response (400):

{ "error": "INVALID_QUALITY" }

TEST B-19: Prompt too long (must fail)

Request: POST <http://localhost:3001/api/generate/image> X-License-Key: poc-test-key-12345678

Body: { "prompt": "<paste 2001 characters here>" }

Expected response (400):

{ "error": "PROMPT_TOO_LONG" }

PART 3 — WEBSITE TESTS

3.1 Open the website

Open file: poc-website/index.html

Double-click the file in Windows Explorer OR drag it into Chrome.

URL in browser will look like: file:///C:/path/to/poc-website/index.html

TEST W-01: Landing page loads

Expected:

- Dark background
 - "YourBrand" logo top left
 - Headline: "Create Anything with AI in Seconds"
 - "Get POC Access — Free" button
 - Feature pills visible at bottom
-

TEST W-02: Navigate to register form

Action: Click "Get POC Access — Free" button

Expected:

- Page transitions to register form
- "Get Your API Key" heading visible

- Name input field visible
 - Email input field visible
 - "Get Access" button visible
 - "Back" link visible
-

TEST W-03: Register with valid data

Action:

1. Type your name in the Name field
2. Type a real email (e.g. "john@example.com")
3. Click "Get Access"

Expected:

- Button shows spinner while loading
- Page transitions to key display
- "You're all set!" heading
- A 32-character hex key shown (e.g. "a1b2c3d4e5f6...")
- Yellow warning: "Save this key. It won't be shown again."
- "Download YourBrand App (Windows)" button

In server terminal you should see: [Auth] New user registered: john@example.com → key: <32hexchars> [HTTP] POST /api/register → 201

TEST W-04: Copy the key

Action: Click the "Copy" button next to the key

Expected:

- Button text changes to "✓ Copied!" with green background
 - After 2 seconds, resets to "Copy"
 - Key is now in your clipboard (try pasting in Notepad to verify)
-

TEST W-05: Register with bad email (validation)

Action:

1. Navigate back to register form
2. Enter "notanemail" in the email field
3. Click "Get Access"

Expected:

- Red error message: "Please enter a valid email address."
 - No network request made (client-side validation)
-

TEST W-06: Register same email twice

Action: Register with the same email you used in TEST W-03

Expected response shows:

- Same key as before (not a new key)
 - "already_existed: true" in the response
-

TEST W-07: Register with no backend running

Action:

1. Stop the backend (Ctrl+C in server terminal)
 2. Try to register Expected:
 - Red error: "Cannot reach server. Make sure the backend is running on port 3001."
 3. Restart the backend: npm start
-

TEST W-08: Back button works

Action: Click "← Back" on the register form

Expected: Returns to landing page

TEST W-09: "Back to home" link on key page

Action: Complete registration → click "← Back to home" on key display page

Expected: Returns to landing page with "Get POC Access" button

PART 4 — ELECTRON APP TESTS

4.1 Start the Electron app

Open a NEW terminal (keep backend running):

```
cd poc-electron
```

```
npm install    (first time only — takes ~2 minutes)
```

```
npm run dev
```

Expected:

- Vite starts and shows: "Local: <http://localhost:5173>"
 - A new desktop window opens titled "YourBrand App"
 - Dark background, logo, "License Key" input field visible
-

TEST E-01: Auth page renders correctly

Expected:

- YourBrand logo (star-like icon in purple gradient square)
 - "YourBrand" title
 - "AI Creative Studio" subtitle
 - "License Key" label
 - Text input with placeholder "Enter your API license key"
 - "Activate" button (disabled when empty)
 - "Don't have a key? Get access at yoursite.com" link at bottom
 - "v0.1.0-poc" version at very bottom
-

TEST E-02: Activate button disabled on empty input

Expected:

- "Activate" button is greyed out when input is empty
 - Clicking it does nothing
-

TEST E-03: Enter invalid key

Action: Type "wrong-key-000" and click Activate

Expected:

- Button shows spinner "Activating..."
 - Red error box appears: "Invalid or expired license key."
 - Input is still visible and editable
-

TEST E-04: Enter key with backend not running

Action:

1. Stop backend (Ctrl+C)
 2. Enter the test key and click Activate Expected:
 - Red error: "Cannot reach server. Make sure the backend is running on port 3001."
 3. Restart backend: cd poc-backend && npm start
-

TEST E-05: Activate with valid test key

Action: Type "poc-test-key-12345678" and click Activate (or press Enter)

Expected:

- Button shows spinner
 - Window transitions to Dashboard page
 - Header shows: "test@poc.local" | "MONTHLY ACCESS" badge | "Xd left" | "Sign out"
 - Center shows: "Choose Your Tool" heading
 - Two cards visible: "AI Image Generation" and "AI Video Generation"
 - Footer: "YourBrand • v0.1.0-poc"
-

TEST E-06: Key persists after restart

Action:

1. Complete TEST E-05 (you're on dashboard)

2. Close the Electron window (X button)
3. Run npm run dev again

Expected:

- App opens directly on Dashboard (skips auth)
 - Same email and subscription visible
 - (Key is saved in localStorage by Zustand persist)
-

TEST E-07: Sign out

Action: Click "Sign out" button in header

Expected:

- Returns to AuthPage
 - Input is empty
 - Key cleared from storage (re-opening app should show auth page)
-

TEST E-08: Navigate to Image Generation

Action: From Dashboard, click "Open" on the Image card

Expected:

- Page changes to Image Generation screen
 - "← Dashboard" back button top left
 - Blue dot + "AI Image Generation" title
 - Dark canvas area with placeholder icon
 - Text: "Start creating or drop media"
 - Bottom bar with: [+] [prompt input] [aspect ratio selector] [send button]
 - Send button is disabled when prompt is empty
-

TEST E-09: Generate an image — MAIN HAPPY PATH

Action:

1. Type: "a glowing forest with fireflies at night"

2. Make sure aspect ratio shows "16:9"
3. Press Enter OR click the arrow send button

Expected sequence: STEP 1 (immediate): Send button shows spinner STEP 2 (0-1s): Canvas shows animated skeleton pulse (loading state) STEP 3 (watch server terminal): "[Worker] Processing job ... | type=image" "[FlowProxy:MOCK] Image — ... (4000ms)" STEP 4 (after ~4 seconds): Canvas shows a real photo from picsum.photos "Done ✓" shows in top right of header "Download Image" button appears below the image

TEST E-10: Download the generated image

Action: Click "Download Image" button

Expected:

- Browser/system download dialog OR file saved automatically
 - File named "yourbrand-image-<timestamp>.jpg"
-

TEST E-11: Generate another image (replaces previous)

Action: Type a new prompt and press Enter

Expected:

- Previous image disappears
 - Skeleton loader appears again
 - New image appears after ~4 seconds
-

TEST E-12: Navigate back to Dashboard

Action: Click "← Dashboard" button

Expected:

- Returns to Dashboard with both tool cards
 - No error, no lost state
-

TEST E-13: Navigate to Video Generation

Action: From Dashboard, click "Open" on the Video card

Expected:

- Video Generation page loads
 - Purple dot + "AI Video Generation" title
 - Empty canvas with video icon
 - Bottom bar: [prompt] [4s/8s buttons] [Fast/HD buttons] [send]
 - Duration buttons: "4s" and "8s" (8s selected by default)
 - Quality buttons: "Fast" (selected) and "HD"
-

TEST E-14: Generate a video — MAIN HAPPY PATH

Action:

1. Type: "ocean waves crashing at sunset"
2. Select "8s" duration
3. Select "Fast" quality
4. Press Enter

Expected sequence:

STEP 1: Send button spins

STEP 2: Canvas shows animated progress indicator

"Generating video..." text

Animated progress bar (blue-to-purple gradient)

"X seconds elapsed" counter ticking up

STEP 3 (after ~8 seconds):

Video appears and auto-plays with controls

"Done in Xs ✓" in header

"Download Video" button visible

TEST E-15: Video player controls work

Action: On the completed video, use player controls

Expected:

- Play/pause works
- Timeline scrubber works
- Volume control works
- Video loops (autoplay loop)

TEST E-16: Download the video

Action: Click "Download Video"

Expected:

- MP4 file downloads or opens
 - Named "yourbrand-video-<timestamp>.mp4"
-

TEST E-17: Duration selector changes

Action:

1. Go to Video page
 2. Click "4s" Expected:
- "4s" button turns purple/highlighted
 - "8s" button deselects
-

TEST E-18: Quality selector changes

Action: Click "HD"

Expected:

- "HD" button highlighted
 - "Fast" deselects
-

TEST E-19: Empty prompt does nothing

Action: Click send with empty prompt

Expected:

- Nothing happens
 - Send button stays disabled
 - No API call made
-

TEST E-20: Navigate away during generation cancels poll

Action:

1. Submit a prompt
 2. While skeleton is showing (within 4 seconds), click "← Dashboard" Expected:
 - Navigates to Dashboard immediately
 - No error in console
 - No "cannot set state on unmounted component" warning
 - (The useEffect cleanup stops the polling)
-

PART 5 — END-TO-END FLOW TEST (all 7 success criteria)

This is the final proof that the POC works.

Run these 7 steps in exact order.

PRE-CONDITION: Backend is running (npm start in poc-backend/)

Redis is running (redis-cli ping → PONG)

Electron app is running (npm run dev in poc-electron/)

STEP 1: Backend starts on port 3001, no errors → Open <http://localhost:3001/health> in browser → See: { "status": "ok", "mode": "mock" } →  PASS if you see this

STEP 2: Register via website → get a key

→ Open poc-website/index.html

→ Click "Get POC Access"

→ Enter name + email → click "Get Access"

→ See a 32-char key

→ Copy it

→  PASS if key appears

STEP 3: Enter key in Electron app → see Dashboard

→ In Electron app, paste the key from Step 2

→ Click Activate

→ See "Choose Your Tool" dashboard

→  PASS if dashboard loads

STEP 4: Generate an image → see skeleton loader

→ Click "Open" on Image card

→ Type: "a beautiful mountain sunrise"

→ Press Enter

→ See skeleton loading animation within 1 second

→  PASS if skeleton appears

STEP 5: Image appears after delay

→ Wait ~4 seconds

→ See a real photo in the canvas

→ Download button appears

→  PASS if image loads

STEP 6: Generate a video → see progress

→ Click "← Dashboard"

→ Click "Open" on Video card

→ Type: "a time-lapse of clouds moving"

→ Press Enter

→ See "Generating video..." with progress bar

→  PASS if progress shows

STEP 7: Video appears and plays

→ Wait ~8 seconds

→ Video appears and plays automatically

→ Controls visible (play, pause, timeline)

→  PASS if video plays

ALL 7 STEPS PASS = POC IS PROVEN 

PART 6 — ERROR AND EDGE CASE TESTS

TEST ERR-01: Start backend without Redis

Action:

1. Stop Redis: redis-cli shutdown
2. Start backend: npm start Expected:
 - Backend starts but shows worker error
 - "[Startup] Worker start error: ..."
 - Server still responds to health check
 - But generate requests return 500 "QUEUE_ERROR"
3. Restart Redis before continuing other tests

TEST ERR-02: Generate while backend is down

Action:

1. Stop backend (Ctrl+C)
2. In Electron app, try to generate an image Expected:
 - Status "failed" appears
 - Error message: "Cannot reach server..."
3. Restart backend

TEST ERR-03: Multiple rapid generates

Action:

1. Quickly submit 3 image prompts one after another (don't wait) Expected:
 - Each gets its own job_id
 - Jobs process one at a time (queue with concurrency=1)
 - Each completes in turn (~4s apart in mock mode)

TEST ERR-04: Very long prompt (near limit)

Action:

1. Type exactly 1999 characters as prompt Expected:
 - Accepted, job created normally

TEST ERR-05: Exactly 2001 character prompt

Expected:

- Error: "Prompt must be under 2000 characters"
-

PART 7 — WHAT EACH LOG LINE MEANS

When you run `npm start`, you'll see these logs. Here's what each means:

[DB] Schema initialized

→ SQLite file created at poc-backend/poc.db

[DB] Seed user already exists, skipping

→ The test user with key poc-test-key-12345678 is already in the DB

[FlowProxy] Mode: MOCK

→ No real AI calls. Mock mode active.

[Queue] Worker started (concurrency=1)

→ Background worker running. Will process 1 job at a time.

[SessionKeeper] Started — pinging Flow every 10 minutes

→ In mock mode this is a no-op. In browser/api mode it keeps session alive.

[HTTP] POST /api/generate/image → 202 (45ms)

→ An image generation request was received and queued.

[Worker] Processing job <id> | type=image → The job left the queue and is now being processed.

[FlowProxy:MOCK] Image — "..." (4000ms) → Mock mode sleeping for 4 seconds to simulate generation.

[Worker] Job <id> completed → <https://picsum.photos/...> → Job done. URL saved to DB.

[Queue] Job bullmq-<id> completed → BullMQ confirmed the job is finished.

PART 8 — CHECKING THE DATABASE DIRECTLY

SQLite file is at: poc-backend/poc.db

If you have a SQLite viewer (DB Browser for SQLite, DBeaver, etc.):

Open poc.db

Table poc_users: should show test user + any registered users

Table poc_jobs: should show all generation jobs with status/output_url

Quick check via Node.js (in poc-backend folder):

```
node -e "const db = require('./db'); console.log(db.getDb().prepare('SELECT * FROM poc_users').all())"
```

```
node -e "const db = require('./db'); console.log(db.getDb().prepare('SELECT id,type,status,output_url FROM poc_jobs').all())"
```

PART 9 — COMMON PROBLEMS AND FIXES

PROBLEM: "Cannot find module 'better-sqlite3'"

FIX: cd poc-backend && npm install

PROBLEM: "QUEUE_ERROR: Is Redis running?"

FIX: redis-server (or start Redis service)

Verify: redis-cli ping → should say PONG

PROBLEM: Electron app white screen

FIX: Wait 10 seconds for Vite to compile

Check npm run dev terminal for errors

PROBLEM: "Error: listen EADDRINUSE :::3001" FIX: Port 3001 is already in use Kill it: (Windows)

netstat -ano | findstr :3001 → taskkill /PID <num> /F (Mac/Linux) lsof -i :3001 → kill -9 <PID>

PROBLEM: Image/video never loads (stuck on skeleton) FIX: Check Redis is running Check server terminal for worker errors Try: curl http://localhost:3001/api/jobs/<job_id> -H "X-License-Key: poc-test-key-12345678"

PROBLEM: Jobs stuck in "queued" status forever

FIX: Worker needs Redis. Redis not running.

Start Redis, then restart the backend.

PROBLEM: "ERR_NETWORK" in Electron

FIX: Backend is not running. cd poc-backend && npm start

PROBLEM: npm install fails on Windows with native module error

FIX: Install Windows Build Tools:

npm install --global windows-build-tools

OR: Install Visual Studio Build Tools

SUMMARY CHECKLISTS

Backend working : ☐ npm start → no errors ☐ GET /health → 200 ok ☐ POST /api/validate-key (test key) → valid: true ☐ POST /api/register → returns license_key ☐ POST /api/generate/image → returns job_id ☐ GET /api/jobs/:id → returns completed with output_url

Website working :

- ☐ Opens in browser
- ☐ Register form works
- ☐ Key displays after registration
- ☐ Copy button works

Electron app working :

- ☐ Opens without errors
- ☐ Test key activates → Dashboard
- ☐ Image generation works (skeleton → image)

- Video generation works (progress → video)
- Sign out returns to auth

All 7 POC success criteria :

- 1. Server starts
- 2. Register on website → get key
- 3. Enter key → Dashboard
- 4. Type prompt → skeleton
- 5. Image appears
- 6. Type video prompt → progress
- 7. Video plays